

# Financial Econometrics Practice TS #3

Daniel Campos, Joshua Castillo, and Cherryl Chico

March 11, 2026

## Contents

|          |                                  |          |
|----------|----------------------------------|----------|
| <b>1</b> | <b>Data Preparation</b>          | <b>1</b> |
| 1.1      | Price and Return Plots . . . . . | 2        |
| <b>2</b> | <b>Volatility Models</b>         | <b>2</b> |
| 2.1      | Fitted Equations . . . . .       | 3        |
| <b>3</b> | <b>Diagnostic Comparison</b>     | <b>4</b> |
| 3.1      | Residual Plots . . . . .         | 5        |
| <b>4</b> | <b>Best Model: GARCH(1,1)</b>    | <b>8</b> |

## 1 Data Preparation

We are tasked to fit different volatility models (ARCH and GARCH) on the Coca-Cola Company stock (KO) stock price from 2007 onward.

```
library(quantmod)
library(fGarch)
library(fBasics)

# Download KO prices from Yahoo Finance starting in 2007.
getSymbols("KO", from = "2007-01-01", auto.assign = TRUE)

# Keep the adjusted closing price and compute continuously compounded returns.
ko_price <- Ad(KO)
ko_return <- na.omit(diff(log(ko_price)))
ko_return_vec <- as.numeric(ko_return)

# Record the sample window for the discussion.
sample_summary <- data.frame(
  start = as.character(start(ko_return)),
  end = as.character(end(ko_return)),
  observations = NROW(ko_return)
)

knitr::kable(sample_summary, caption = "Sample used in the analysis")

## [1] "KO"
```

Table 1: Sample used in the analysis

| start      | end        | observations |
|------------|------------|--------------|
| 2007-01-04 | 2026-03-12 | 4827         |

## 1.1 Price and Return Plots

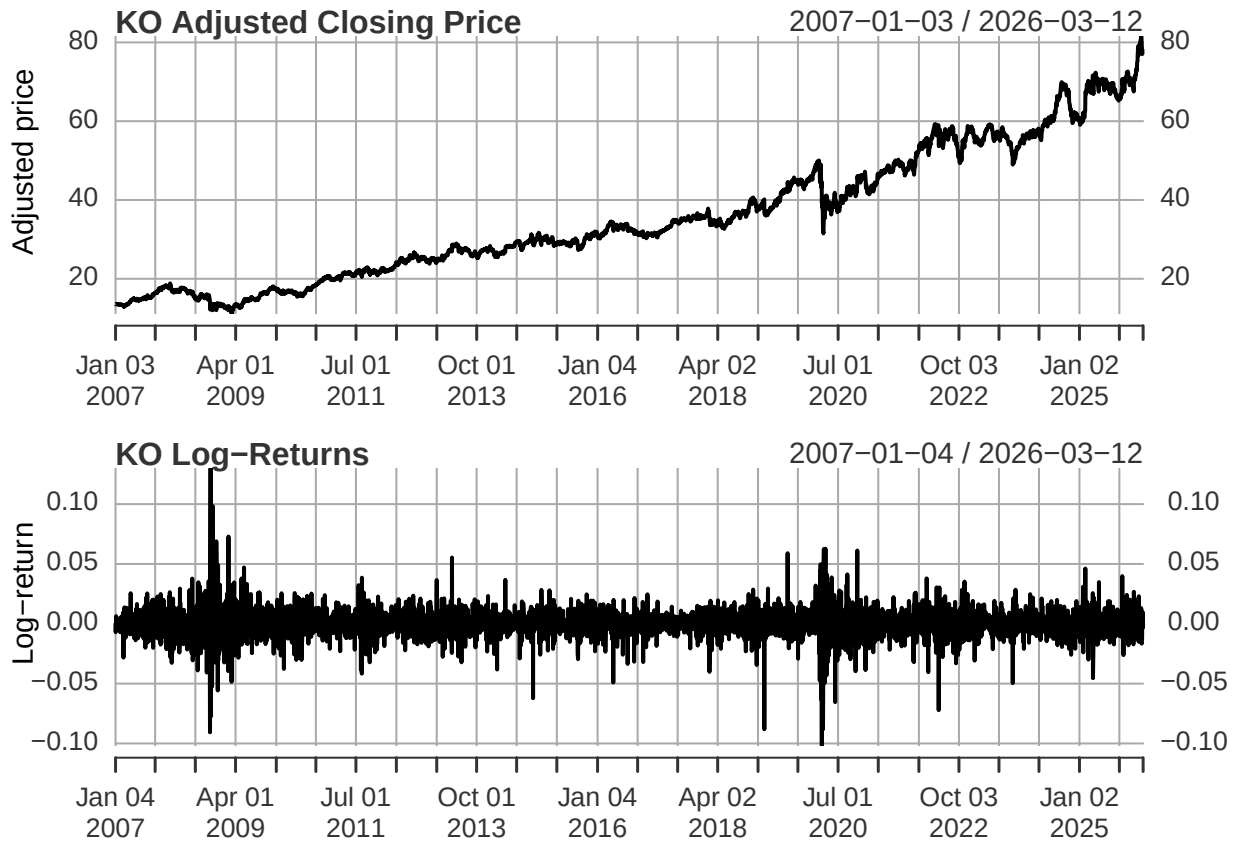
The adjusted price shows the long-run trend in the stock, while log-returns make volatility clustering visible. The returns fluctuate around a small mean, but large and small movements clearly come in clusters, which motivates the use of ARCH/GARCH models.

```
par(mfrow = c(2, 1), mar = c(3, 4, 2, 1))

# Plot the adjusted closing price.
plot(ko_price, main = "KO Adjusted Closing Price", ylab = "Adjusted price", xlab = "")

# Plot the log-returns used for estimation.
plot(ko_return, main = "KO Log-Returns", ylab = "Log-return", xlab = "Date")

par(mfrow = c(1, 1))
```



## 2 Volatility Models

The assignment asks for three variance specifications. Since the mean is not modeled with ARIMA terms here, the fitted equations take the form

$$r_t = \mu + Z_t, \quad Z_t = \sigma_t \varepsilon_t, \quad \varepsilon_t \sim N(0, 1),$$

with a different law for  $\sigma_t^2$  in each case.

```
# Fit the three volatility models requested in the questionnaire.
fit_arch1 <- garchFit(~ garch(1, 0), data = ko_return_vec, trace = FALSE)
fit_arch3 <- garchFit(~ garch(3, 0), data = ko_return_vec, trace = FALSE)
fit_garch11 <- garchFit(~ garch(1, 1), data = ko_return_vec, trace = FALSE)

fits <- list(
  "ARCH(1)" = fit_arch1,
  "ARCH(3)" = fit_arch3,
  "GARCH(1,1)" = fit_garch11
)
```

## 2.1 Fitted Equations

The estimated coefficients below give the explicit fitted model for each specification. We report the mean equation and the conditional variance equation separately so the interpretation is clear.

```
write_equation <- function(fit, label) {
  cf <- coef(fit)

  cat("\n### ", label, "\n\n", sep = "")
  cat(sprintf(
    "$r_t = %.6f + Z_t, \\quad Z_t = \\sigma_t \\varepsilon_t, \\quad \\varepsilon_t \\sim N(0,1)$\\n",
    cf["mu"]
  ))
  if (label == "ARCH(1)") {
    cat(sprintf(
      "$\\sigma_t^2 = %.8f + %.6f Z_{t-1}^2.$\\n\\n",
      cf["omega"], cf["alpha1"]
    ))
  }
  if (label == "ARCH(3)") {
    cat(sprintf(
      "$\\sigma_t^2 = %.8f + %.6f Z_{t-1}^2 + %.6f Z_{t-2}^2 + %.6f Z_{t-3}^2.$\\n",
      cf["omega"], cf["alpha1"], cf["alpha2"], cf["alpha3"]
    ))
  }
  if (label == "GARCH(1,1)") {
    cat(sprintf(
      "$\\sigma_t^2 = %.8f + %.6f Z_{t-1}^2 + %.6f \\sigma_{t-1}^2.$\\n\\n",
      cf["omega"], cf["alpha1"], cf["beta1"]
    ))
    cat(sprintf(
      "The persistence measure is $\\alpha_1 + \\beta_1 = %.6f$, which indicates highly persistent volatility.\\n",
      cf["alpha1"] + cf["beta1"]
    ))
  }
}

invisible(mapply(write_equation, fits, names(fits)))
```

### 2.1.1 ARCH(1)

$$r_t = 0.000616 + Z_t, \quad Z_t = \sigma_t \varepsilon_t, \quad \varepsilon_t \sim N(0, 1) \\ \sigma_t^2 = 0.00009043 + 0.341198 Z_{t-1}^2.$$

### 2.1.2 ARCH(3)

$$r_t = 0.000542 + Z_t, \quad Z_t = \sigma_t \varepsilon_t, \quad \varepsilon_t \sim N(0, 1) \\ \sigma_t^2 = 0.00006719 + 0.129717 Z_{t-1}^2 + 0.187096 Z_{t-2}^2 + 0.146654 Z_{t-3}^2.$$

### 2.1.3 GARCH(1,1)

$$r_t = 0.000494 + Z_t, \quad Z_t = \sigma_t \varepsilon_t, \quad \varepsilon_t \sim N(0, 1) \\ \sigma_t^2 = 0.00000318 + 0.066554 Z_{t-1}^2 + 0.907538 \sigma_{t-1}^2.$$

The persistence measure is  $\alpha_1 + \beta_1 = 0.974093$ , which indicates highly persistent volatility.

## 3 Diagnostic Comparison

To choose the best specification, we compare information criteria and residual diagnostics. The most important checks are whether standardized residuals behave like white noise and whether squared standardized residuals still show autocorrelation. If squared residual autocorrelation remains, then the volatility model is still missing ARCH effects.

```
diagnostic_table <- do.call(
  cbind,
  lapply(names(fits), function(model_name) {
    fit <- fits[[model_name]]
    epsilon <- as.numeric(residuals(fit, standardize = TRUE))

    c(
      AIC = fit@fit$ics["AIC"],
      BIC = fit@fit$ics["BIC"],
      LB_resid_10_p = Box.test(epsilon, lag = 10, type = "Ljung-Box")$p.value,
      LB_resid_20_p = Box.test(epsilon, lag = 20, type = "Ljung-Box")$p.value,
      LB_sq_10_p = Box.test(epsilon^2, lag = 10, type = "Ljung-Box")$p.value,
      LB_sq_20_p = Box.test(epsilon^2, lag = 20, type = "Ljung-Box")$p.value,
      Skewness = basicStats(epsilon)["Skewness", 1],
      Kurtosis = basicStats(epsilon)["Kurtosis", 1]
    )
  })
)

colnames(diagnostic_table) <- names(fits)

diagnostic_table <- round(diagnostic_table, 4)

diagnostic_table <- data.frame(
  Diagnostic = rownames(diagnostic_table),
  diagnostic_table,
  row.names = NULL,
  check.names = FALSE
)
```

```
knitr::kable(
  diagnostic_table,
  format = "latex",
  booktabs = TRUE,
  longtable = TRUE,
  caption = "Model comparison using standardized residual diagnostics"
)
```

Table 2: Model comparison using standardized residual diagnostics

| Diagnostic    | ARCH(1) | ARCH(3) | GARCH(1,1) |
|---------------|---------|---------|------------|
| AIC.AIC       | -6.2008 | -6.2902 | -6.3275    |
| BIC.BIC       | -6.1968 | -6.2835 | -6.3221    |
| LB_resid_10_p | 0.0113  | 0.0498  | 0.0573     |
| LB_resid_20_p | 0.0639  | 0.1443  | 0.2482     |
| LB_sq_10_p    | 0.0000  | 0.1470  | 0.9461     |
| LB_sq_20_p    | 0.0000  | 0.0140  | 0.9988     |
| Skewness      | -0.1355 | -0.3770 | -0.4078    |
| Kurtosis      | 6.2797  | 5.7565  | 5.5802     |

The table shows a clear ranking. ARCH(1) leaves very strong dependence in the squared standardized residuals, with Ljung-Box p-values essentially equal to zero at both lag 10 and lag 20. ARCH(3) is a substantial improvement, and by lag 10 it no longer rejects, but the squared residual test at lag 20 still rejects at the 5% level.

GARCH(1,1) provides the best fit. It has the lowest AIC and BIC, and its Ljung-Box p-values for squared standardized residuals are very high at both lag choices, which means there is no evidence of remaining ARCH effects. This is exactly the pattern we want from a good volatility specification.

### 3.1 Residual Plots

To make diagnostics directly comparable, each candidate model is evaluated with the same four plots used later for the selected model: standardized residual path, QQ plot, ACF of squared residuals, and PACF of squared residuals.

```
for (model_name in names(fits)) {
  fit <- fits[[model_name]]
  epsilon <- as.numeric(residuals(fit, standardize = TRUE))

  # Increase top/bottom margins so row 1 and row 2 panels do not look crowded.
  par(mfrow = c(2, 2), mar = c(5, 4, 3, 1))

  # Standardized residual path.
  plot(epsilon, type = "l", main = paste(model_name, "standardized residuals"),
       ylab = "epsilon_t", xlab = "Index")
  abline(h = c(-3, 3), col = "firebrick", lty = 2)

  # QQ plot to assess departures from normality.
  qqnorm(epsilon, main = paste(model_name, "QQ plot"))
  qqline(epsilon, col = "steelblue", lwd = 2)

  # Squared residual autocorrelation should be weak if volatility is well captured.
```

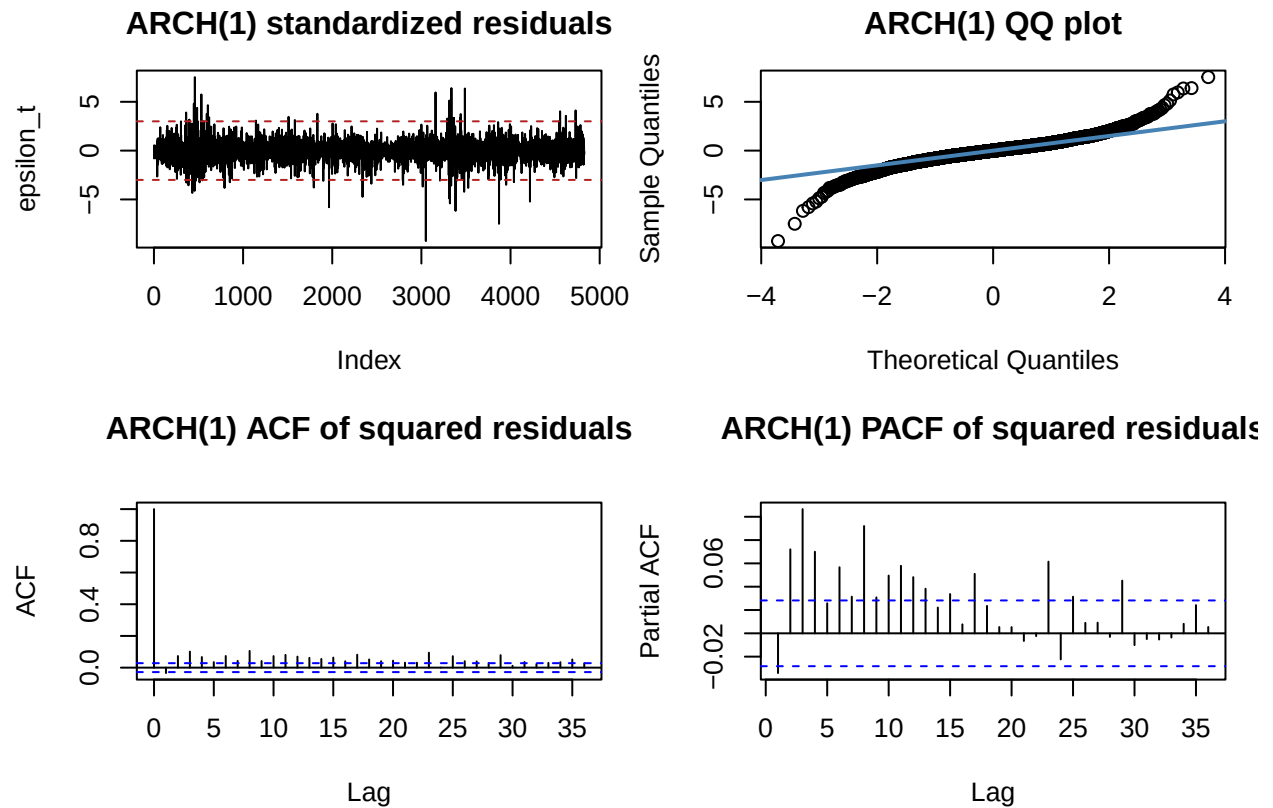
```

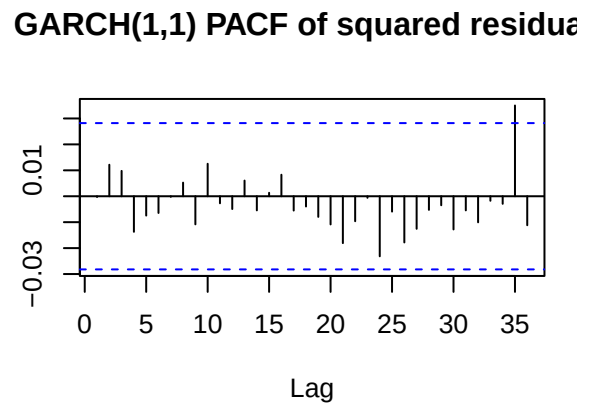
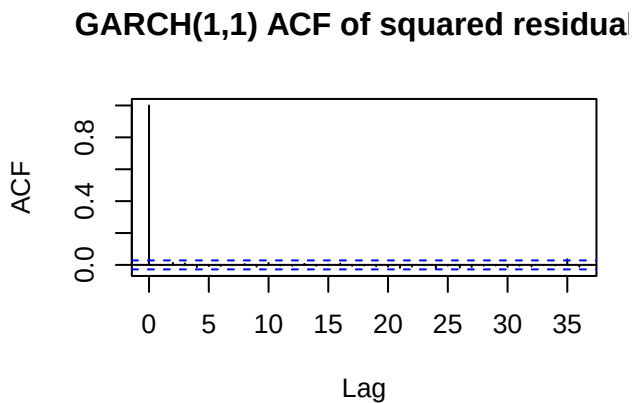
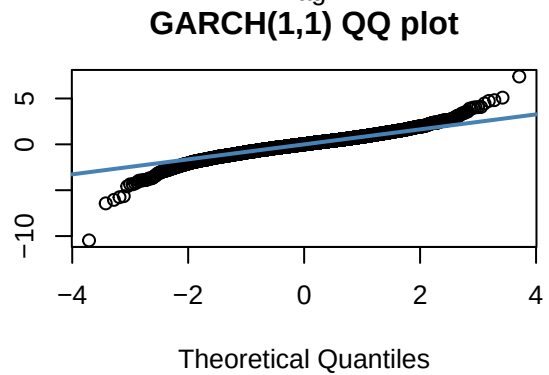
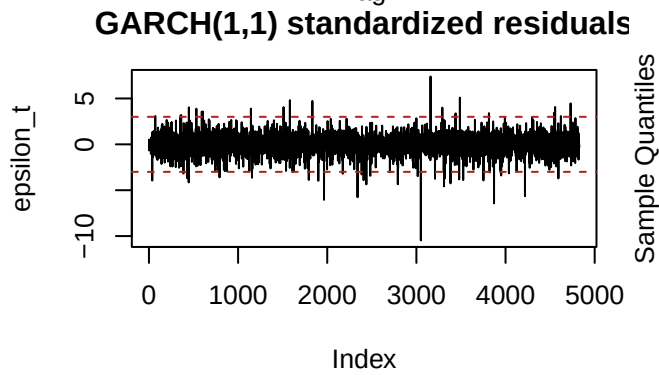
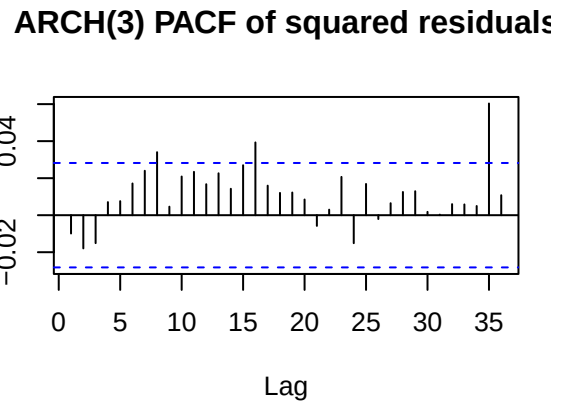
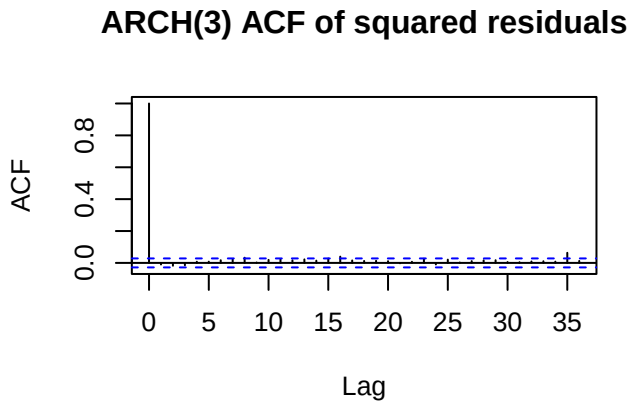
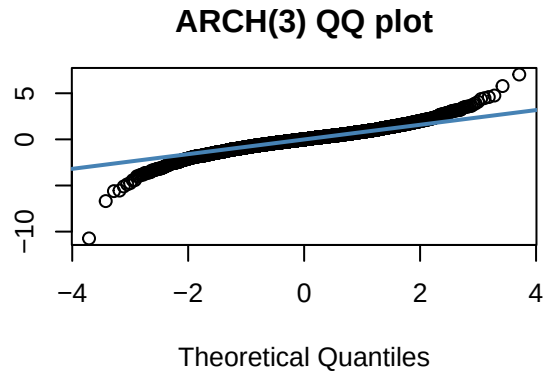
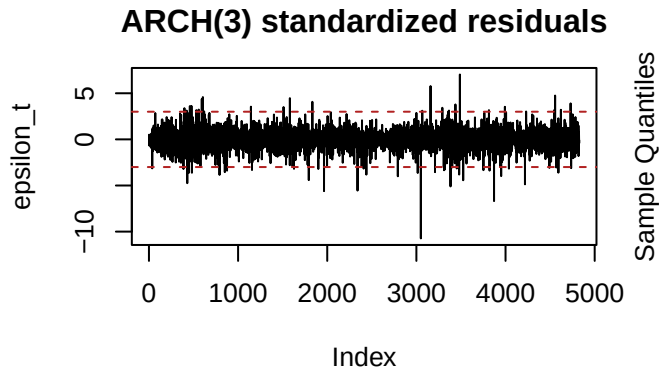
acf(epsilon^2, main = paste(model_name, "ACF of squared residuals"))

# Squared residual PACF highlights any remaining short-lag dependence.
pacf(epsilon^2, main = paste(model_name, "PACF of squared residuals"))
}

par(mfrow = c(1, 1))

```





The residual plots are consistent with the model comparison table. In the standardized residual series, outliers remain for all three models, but they are visibly less pronounced under ARCH(3) and especially GARCH(1,1). The ACF of squared residuals no longer shows explosive behavior for any specification. The

PACF gives a clearer ranking: ARCH(1) still shows many significant lags, ARCH(3) shows fewer, and GARCH(1,1) performs best, with no meaningful significant lag until around lag 35, which is not a practical concern here. The QQ plots also confirm that normality is still not fully achieved, but GARCH(1,1) is closest to the theoretical quantiles in the tails and has the lowest kurtosis among the three models.

## 4 Best Model: GARCH(1,1)

Based on the diagnostic table and residual-plot comparison above, we select **GARCH(1,1)** as the best model among the three candidates. To avoid repeating the same diagnostics, we report only the final estimated equation here.

```
cf_best <- coef(fit_garch11)
```

```
cat(sprintf(
  "$r_t = %.6f + Z_t, \\quad Z_t = \\sigma_t \\varepsilon_t,
  \\quad \\varepsilon_t \\sim N(0,1)\\n\\n",
  cf_best["mu"]
))
```

$$r_t = 0.000494 + Z_t, \quad Z_t = \sigma_t \varepsilon_t, \quad \varepsilon_t \sim N(0, 1)$$

```
cat(sprintf(
  "$\\sigma_t^2 = %.8f + %.6f Z_{t-1}^2 + %.6f \\sigma_{t-1}^2.\\n\\n",
  cf_best["omega"], cf_best["alpha1"], cf_best["beta1"]
))
```

$$\sigma_t^2 = 0.00000318 + 0.066554 Z_{t-1}^2 + 0.907538 \sigma_{t-1}^2.$$

```
cat(sprintf(
  "The persistence measure is $\\alpha_1 + \\beta_1 = %.6f$.\\n\\n",
  cf_best["alpha1"] + cf_best["beta1"]
))
```

The persistence measure is  $\alpha_1 + \beta_1 = 0.974093$ .